TradingAI macOS Setup and Installation Guide

Complete clean-machine installation, validation and current LaunchAgent configuration.

System reference date: 2026-08-29

Standalone current-state reference for the complete platform through Milestone 78.

1. Installation target

This guide installs the current TradingAI/TradingPlatform platform on a clean macOS machine for local paper-trading/development operation. Use a stable project path such as ~/TradingPlatform; do not run long-term services directly from Downloads. Obtain your own Polygon API key, Alpha Vantage key where earnings automation is used, and an Interactive Brokers Paper account with TWS or IB Gateway.

> Never copy credentials from another machine into source-controlled files. The final environment must remain PAPER until separate live-trading certification is explicitly completed.

2. Install Apple command-line tools and Homebrew

```
xcode-select --install

# Install Homebrew only if brew is not already available
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
brew update
brew doctor
```

3. Install required local services and tools

```
brew install uv postgresql@17 redis git jq node

echo 'export PATH="$(brew --prefix postgresql@17)/bin:$PATH"' >> ~/.zshrc
source ~/.zshrc

uv --version
psql --version
redis-server --version
node --version
npm --version
```

4. Start PostgreSQL and Redis

```
brew services start postgresql@17
brew services start redis

pg_isready -h localhost -p 5432
redis-cli ping
brew services list | egrep 'postgresql@17|redis'
```

5. Place the current codebase

```
mkdir -p ~/TradingPlatform
# Copy/extract the CONTENTS of the current TradingPlatform archive into ~/TradingPlatform
cd ~/TradingPlatform
ls pyproject.toml src/trading_ai ui/workstation migrations
```

6. Install Python dependencies

```
cd ~/TradingPlatform
uv sync --all-groups
uv run python --version
uv run python -c "import trading_ai; print(trading_ai.__file__)"
uv run python -m trading_ai --help
```

7. Create the PostgreSQL database

```
createdb trading_ai 2>/dev/null || true
psql -d trading_ai -c 'SELECT current_database(), current_user;'
```

8. Configure environment and credentials

Create a local .env file and populate the variable names expected by the current configuration/provider modules. At minimum the environment must resolve the PostgreSQL connection, Polygon provider credential, Alpha Vantage credential if earnings calendar automation is enabled, PAPER runtime environment and IBKR host/port/client IDs. Do not invent old variable names if they are not present in the current code.

```
cd ~/TradingPlatform
[ -f .env.example ] && cp -n .env.example .env || touch .env
chmod 600 .env
${EDITOR:-nano} .env
```

9. Apply the complete database schema

```
cd ~/TradingPlatform
uv run alembic current
uv run alembic upgrade heads
uv run alembic heads
```

For the post-M78 system, the expected observed Alembic head is **m78_001**. Use heads rather than assuming a single historical head name.

```
uv run python - <<'PY'
from trading_ai.database.session import SessionLocal
s = SessionLocal()
try:
print(s.get_bind())
finally:
s.close()
PY
```

Use SessionLocal() and session.get_bind() or current repository utilities. Do not resurrect obsolete imports such as DATABASE_URL from trading_ai.database.

10. Install and validate the workstation

```
cd ~/TradingPlatform/ui/workstation
npm ci
npm test
npm run typecheck
npm run build
cd ~/TradingPlatform
```

11. Backend regression and M78 focused validation

```
cd ~/TradingPlatform
uv run python -m trading_ai local-doctor || true
uv run python scripts/local_setup_check.py || true
uv run python -m trading_ai --help

# Broad regression suite
uv run pytest -q

# M78 release/focused verification
uv run python scripts/verify_m78_release.py
uv run python -m pytest -q tests/m78
uv run python -m py_compile src/trading_ai/setup_intelligence/*.py scripts/run_m78_setup_intelligence.py
scripts/run_m78_daily_shadow.py
```

M78 verification should report authority_effect=false and production behavior unchanged. Training may legitimately return INSUFFICIENT_EVIDENCE on a new installation; that is not an installation failure.

12. Configure Polygon and validate market data

```
cd ~/TradingPlatform
uv run python test_poly_conn.py
uv run python test_poly_option_chain.py
```

The platform's current provider policy uses Polygon for option-chain/quote data and the current source configuration for underlying market data. Validate the actual provider modules in the installed baseline rather than assuming credentials work.

13. Install and configure IBKR Paper

- Install Interactive Brokers TWS or IB Gateway.
- Log into the Paper Trading environment, not Live.
- Enable API/socket clients and restrict trusted IPs to localhost unless remote access is intentionally designed.
- Configure the paper API host/port/client IDs in TradingAI.
- Do not store broker passwords in TradingAI broker-account binding.
- Verify PAPER-PRIMARY maps to the intended paper account before any order workflow.

14. First controlled startup

```
cd ~/TradingPlatform
./start_platform.sh --help || true
./platform_status.sh
uv run python -m trading_ai start --mode paper --dry-run || true
./start_platform.sh
./platform_status.sh
```

If readiness reports a missing dependency or stale schema, fix the prerequisite; do not bypass startup governance.

15. Start the workstation

```
cd ~/TradingPlatform/ui/workstation
npm run dev
```

The Vite development server normally serves the workstation locally. Use the URL reported by Vite; do not hard-code a port if the local configuration changes it.

16. Seed intelligence in governed dependency order

A new database needs market observations and derived publications before downstream pages can become READY. Preserve the dependency order: underlying/market -> trend -> Stock Intelligence -> options/valuation -> Decision Intelligence -> portfolio -> Trade Builder/execution.

```
cd ~/TradingPlatform
uv run python scripts/ingest_underlying_data.py --help
uv run python scripts/ingest_options_data.py --help
uv run python scripts/run_market_ingestion.py --help 2>/dev/null || true
```

17. Install current LaunchAgents only after manual validation

Do not blindly run every historical installer in the repository. Install only the current authoritative service for each responsibility, then verify with launchctl.

LaunchAgent	Schedule	Purpose
com.tradingplatform.m71-futures-preopen	Mon-Fri 07:55	Futures pre-open intelligence.
com.tradingplatform.m69-6-event-intelligence	Mon-Fri 08:10	Event intelligence.
com.tradingplatform.m69-6-morning-ingestion	Mon-Fri 08:30	Morning ingestion.
com.tradingplatform.m69-6-intraday-options	Mon-Fri hourly 09:30-14:30	Intraday options intelligence.
com.tradingplatform.m69-6-end-of-day-ingestion	Mon-Fri 15:20	End-of-day ingestion.
com.tradingplatform.m77-m78-daily-shadow	Mon-Fri 18:30	Ordered M77 -> M78 research/shadow.

18. Verify LaunchAgent registration

```
launchctl list | grep -Ei 'tradingplatform|tradingai'
launchctl print gui/$(id -u)/com.tradingplatform.m77-m78-daily-shadow
```

For user LaunchAgents in ~/Library/LaunchAgents, use the gui/$(id -u) domain and do not use sudo. If a bulk reload returns Bootstrap failed: 5, validate the plist, bootout the specific label, wait briefly, bootstrap that plist individually, then verify with launchctl print.

19. Verify M78 operation

```
cd ~/TradingPlatform
uv run python scripts/run_m78_setup_intelligence.py capture
uv run python scripts/run_m78_setup_intelligence.py materialize-outcomes
uv run python scripts/run_m78_setup_intelligence.py status
```

Do not train until readiness says evidence is sufficient. A trained challenger still requires explicit approve-shadow and activate-shadow actions, and remains shadow-only.

20. Paper-trading safety gate before first order

- Runtime environment is PAPER and the broker account is the intended IBKR paper account.
- Live-trading governance remains disabled unless separately certified.
- Current Stock Intelligence, Decision Intelligence and portfolio risk/allocation publications are READY and lineage-consistent.
- Trade Builder produces a current certified plan.
- Execution preflight passes freshness, drift, liquidity and broker-tick checks.
- Broker reconciliation is healthy.
- Autonomous position management is installed/running before relying on automatic exits.
- Expiration governance is active: close before expiration, using the earliest leg expiration for multi-leg positions.

21. Clean-machine acceptance checklist

- PostgreSQL and Redis healthy under brew services.
- uv sync succeeds with Python >=3.13.
- Alembic is at m78_001 and the database is reachable through SessionLocal.
- Backend regression and tests/m78 pass or have explained environment-dependent failures.
- Frontend npm ci/test/typecheck/build succeeds.
- Polygon connectivity succeeds.
- IBKR Paper connectivity/account mapping succeeds.
- Manual ingestion creates current Market/Trend/Stock/Options authorities.
- API/workstation load without schema/runtime errors.
- Broker portfolio synchronization produces reconciled current state.
- Trade Builder revalidation and execution preflight remain paper-only.
- Expected LaunchAgents are loaded with the current Monday-Friday schedules.

22. Troubleshooting quick reference

```
# PostgreSQL
pg_isready -h localhost -p 5432
brew services list
psql trading_ai -c 'select now();'

# Migrations
cd ~/TradingPlatform
uv run alembic current
uv run alembic heads

# Package
uv run python -c 'import trading_ai; print(trading_ai.__file__)'
```

```
# Platform
./platform_status.sh
ps aux | grep -E 'trading_ai|uvicorn|vite' | grep -v grep

# LaunchAgents
launchctl list | grep -E 'tradingplatform|tradingai'

# UI
cd ~/TradingPlatform/ui/workstation
npm test
npm run typecheck
npm run build
```